# LLMOps Multi-Agent Platform for Prompt Refinement using Google Gemini

PROJECT DOCUMENTATION

**Submitted By**

K. Lasya
2451-23-749-017
Computer Science and Engineering (Allied)

**College**
Maturi Venkata Subba Rao Engineering College

Academic Year
2026–2027

# Contents

# 1. Abstract

Artificial Intelligence has become an important part of our daily lives, helping people generate text, write code, answer questions, and solve problems. Large Language Models (LLMs) such as GPT, Gemini, and Llama have made these tasks much easier. However, the quality of their responses depends largely on the prompt given by the user. If the prompt is unclear, incomplete, or poorly written, the AI may produce inaccurate or irrelevant results. Many users are not familiar with prompt engineering, making it difficult for them to obtain the best possible output from AI models.

This project proposes an LLMOps Multi-Agent Platform that automatically improves user prompts before they are sent to a Generative AI model. Instead of relying on users to create perfect prompts, the system uses multiple intelligent agents that work together to understand the user's request, identify missing details, improve clarity, correct grammar, and optimize the prompt. Once the prompt has been refined, it is sent to the Large Language Model, which generates a more accurate and meaningful response.

The platform follows the principles of LLMOps by managing and optimizing interactions between users and Large Language Models. It records prompt history and execution logs for future analysis and continuous improvement. The project demonstrates how prompt engineering, multi-agent systems, and modern Generative AI technologies can work together to improve the quality of AI-assisted applications.

# 2. Existing System

Traditional Generative AI applications directly forward the user's prompt to the Large Language Model without performing any preprocessing or validation. As a result, vague, ambiguous, or incomplete prompts frequently lead to low-quality responses.

Users often need to rewrite their prompts multiple times before obtaining satisfactory results. This trial-and-error process requires knowledge of prompt engineering techniques and increases the time required to complete tasks.

## Limitations

- No automatic prompt refinement.
- Poor handling of ambiguous prompts.
- Depends entirely on user prompt quality.
- Requires multiple iterations to obtain good responses.
- No monitoring or prompt history management.

# 3. Proposed System

The proposed LLMOps Multi-Agent Platform automatically improves user prompts before they are submitted to Google Gemini. Instead of depending solely on the user's ability to write effective prompts, the platform employs multiple intelligent agents that work together to improve prompt quality.

The workflow begins when the user submits a prompt through the web interface. The Validation Agent checks whether the prompt is valid. The Analysis Agent identifies ambiguity, missing context, and grammatical issues. The Refinement Agent rewrites the prompt while preserving the user's intent. Finally, the optimized prompt is sent to Google Gemini for response generation.

The modular architecture makes the platform scalable, reusable, and suitable for future enhancements such as multi-model support and advanced monitoring.

# 4. System Modules

The proposed LLMOps Multi-Agent Platform consists of several modules that work together to improve user prompts before sending them to the Google Gemini model.

## 4.1 User Interface Module

The User Interface provides a simple and interactive environment where users can enter prompts and receive AI-generated responses. It acts as the communication layer between the user and the backend system.

## 4.2 Prompt Validation Agent

This agent verifies whether the prompt entered by the user is valid. It checks for empty inputs and ensures that the prompt is suitable for further processing.

## 4.3 Prompt Analysis Agent

The Prompt Analysis Agent analyzes the prompt to determine the user's intent. It detects grammatical errors, ambiguous statements, and missing contextual information that may affect the quality of the generated response.

## 4.4 Prompt Refinement Agent

The Prompt Refinement Agent rewrites the prompt while preserving its original meaning. It improves grammar, clarity, completeness, and overall structure to maximize the effectiveness of the Large Language Model.

## 4.5 Response Generation Module

The refined prompt is submitted to Google Gemini through its API. The model generates an accurate and context-aware response based on the optimized prompt.

## 4.6 Monitoring and Logging Module

This module records prompt history, refined prompts, execution time, and generated responses. These logs help monitor system performance and support future improvements.

# 5. Algorithm

## 5.1 Objective

The primary objective of the proposed system is to improve user prompts before sending them to a Large Language Model. The system automatically analyzes, refines, validates, and optimizes prompts to obtain better AI-generated responses.

## 5.2 Algorithm Steps

1. Accept the prompt entered by the user.
2. Validate whether the prompt is empty.
3. Analyze the prompt to identify:
    - User intent
    - Grammar issues
    - Missing context
    - Ambiguous statements
4. Refine the prompt by:
    - Correcting grammar
    - Improving clarity
    - Adding necessary details
    - Preserving user intent
5. Validate the refined prompt.
6. If validation fails, repeat the refinement process.
7. Send the optimized prompt to Google Gemini.
8. Generate the AI response.

9. Store execution logs.
10. Display the refined prompt and AI-generated response.

## 5.3 Pseudocode

```
START

Receive Prompt

IF Prompt is Empty

    Display Error Message

ELSE

    Analyze Prompt

    Refine Prompt

    WHILE Prompt is Invalid

        Refine Prompt Again

    END WHILE

    Send Prompt to Google Gemini

    Generate Response

    Save Logs

    Display Output

END
```

## 5.4 Project Flow

```
User Prompt
      |
      V
Validation Agent
      |
      V
Analysis Agent
      |
      V
Refinement Agent
      |
      V
Google Gemini API
      |
      V
Generated Response
      |
      V
Display Output
      |
      V
Store Logs
```

## 5.5 Tools and Technologies

| Technology | Purpose |
|---|---|
| Python | Programming Language |
| Flask | Backend Framework |
| HTML | User Interface |
| CSS | Styling |
| JavaScript | Client-side Interaction |
| Google Gemini API | Response Generation |
| VS Code | Development Environment |
| Git | Version Control |

Table 5.1: Technologies Used

## 5.6 Tools and Technologies

| Technology | Purpose |
|---|---|
| Python | Programming Language |
| Flask | Backend Framework |
| HTML | User Interface |
| CSS | Styling |
| JavaScript | Client-side Interaction |
| Google Gemini API | Response Generation |
| VS Code | Development Environment |
| Git | Version Control |

Table 5.2: Technologies Used

# 6. Results

The developed LLMOps Multi-Agent Platform successfully validates, analyzes, and refines user prompts before generating responses using Google Gemini. The screenshots below demonstrate the successful execution of the proposed system.

## 6.1 Output Screenshots

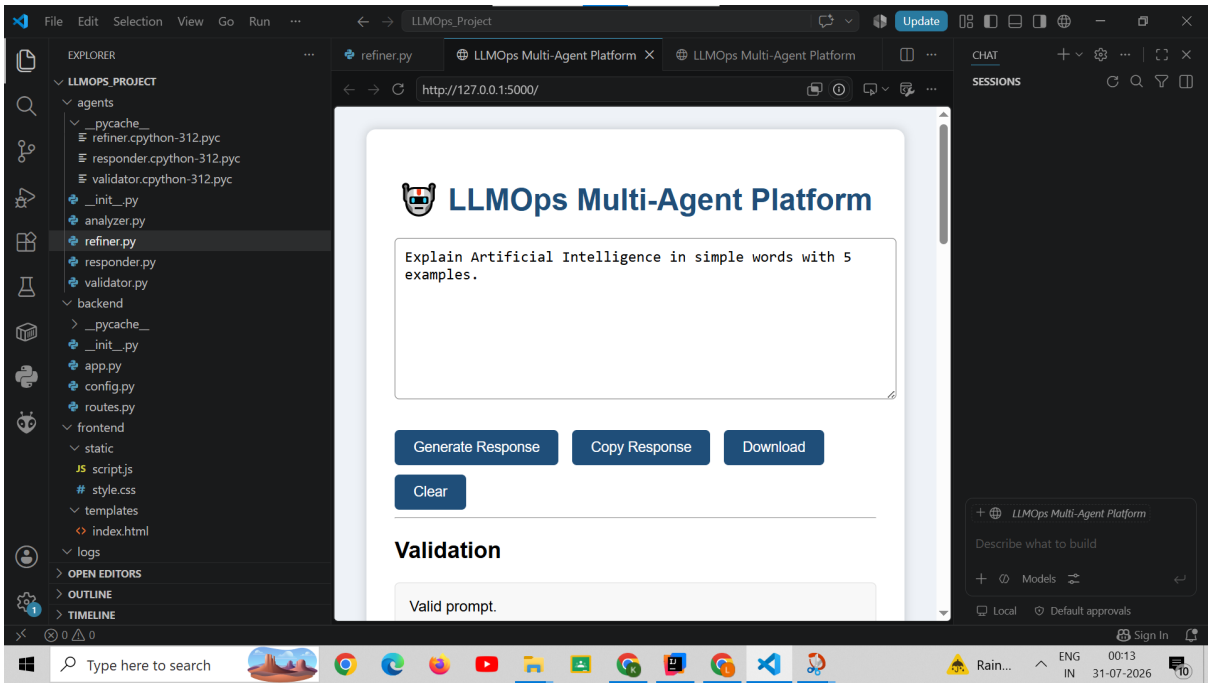


Figure 6.1: Home Page

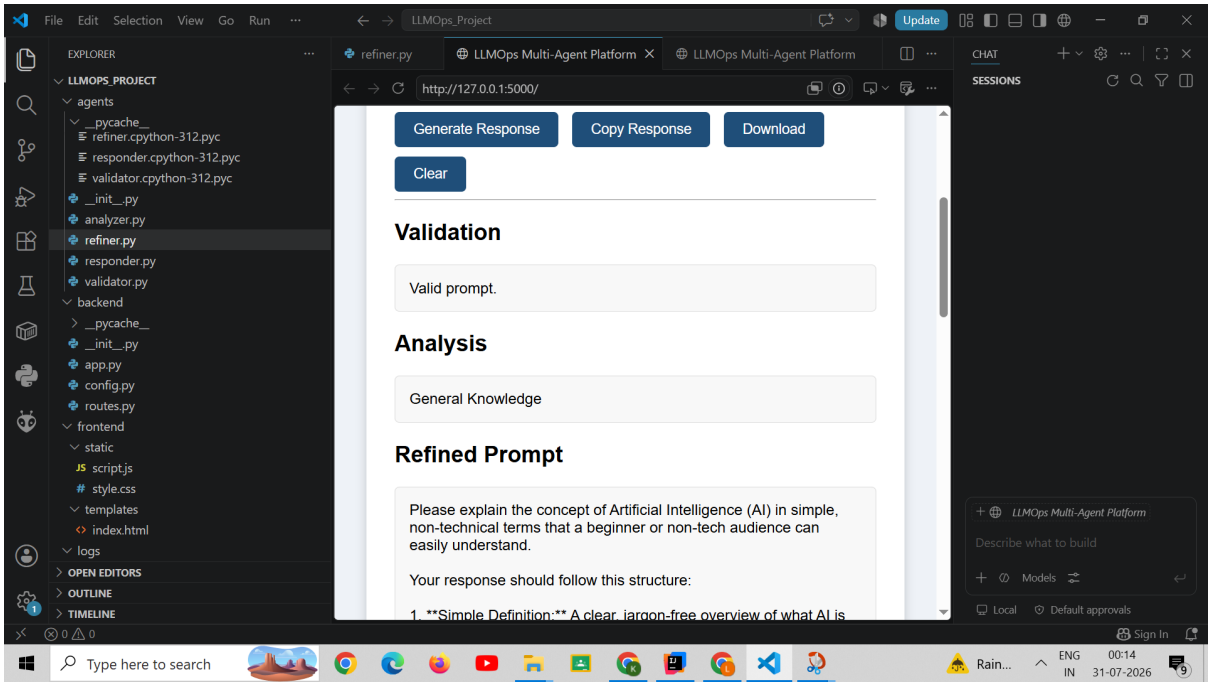


Figure 6.2: Validation Result

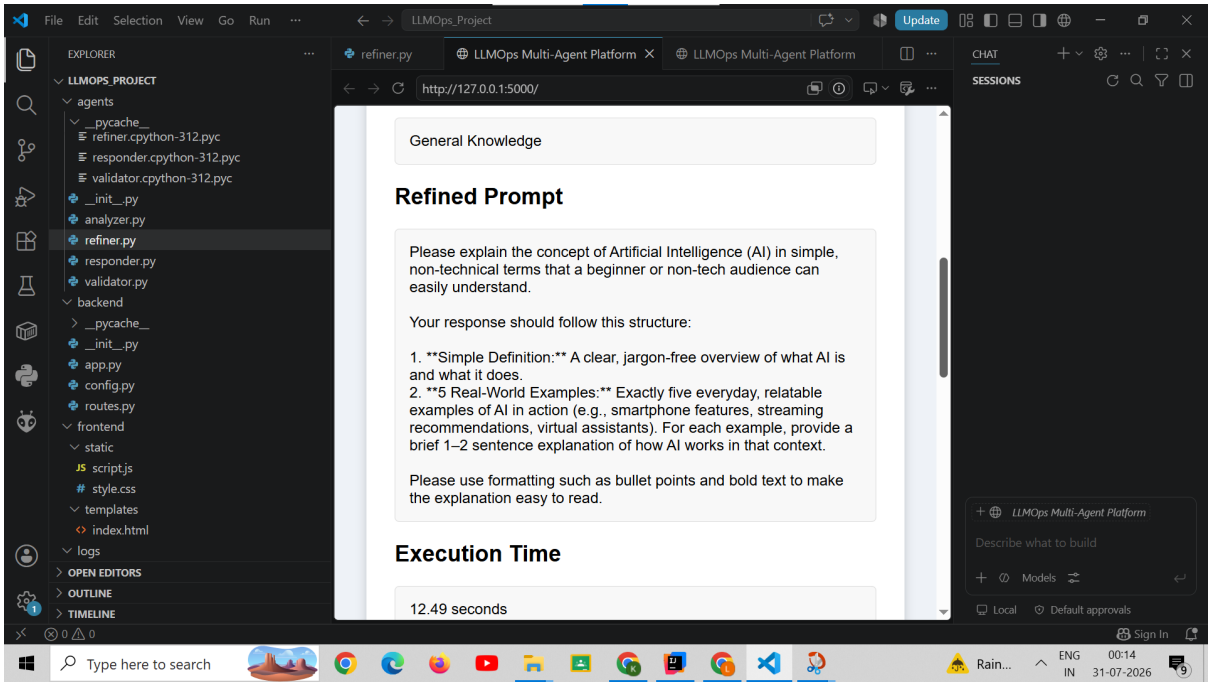


Figure 6.3: Prompt Refinement

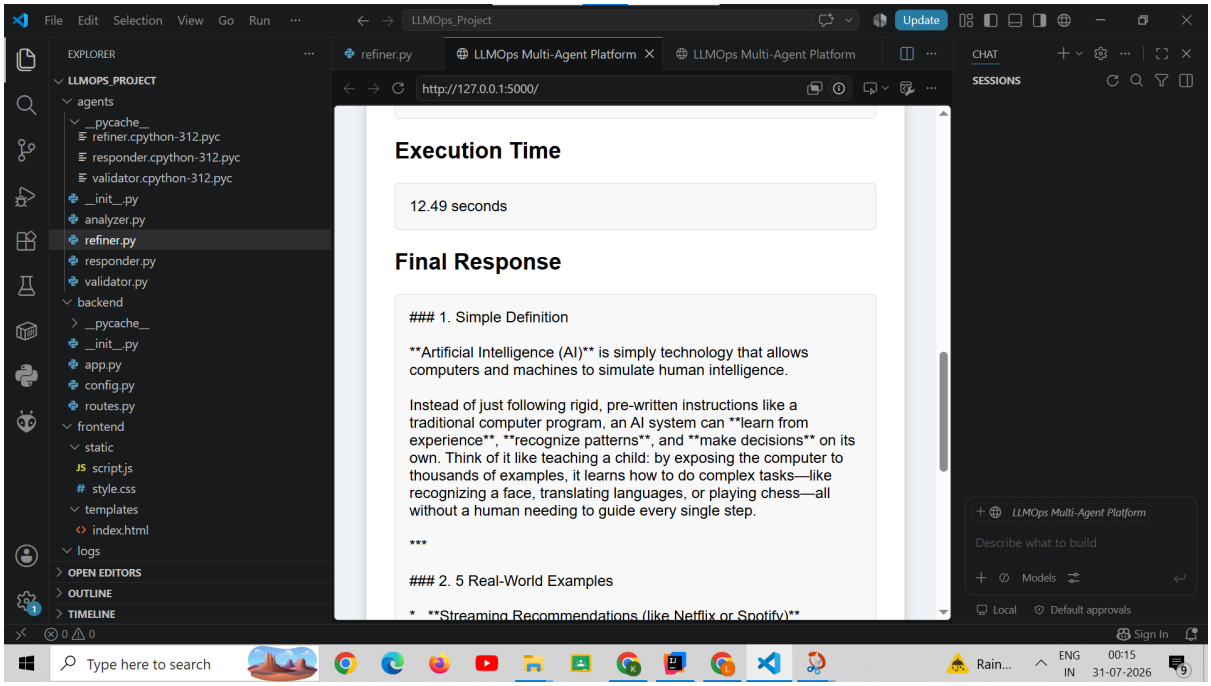


Figure 6.4: Execution Time and Final Response

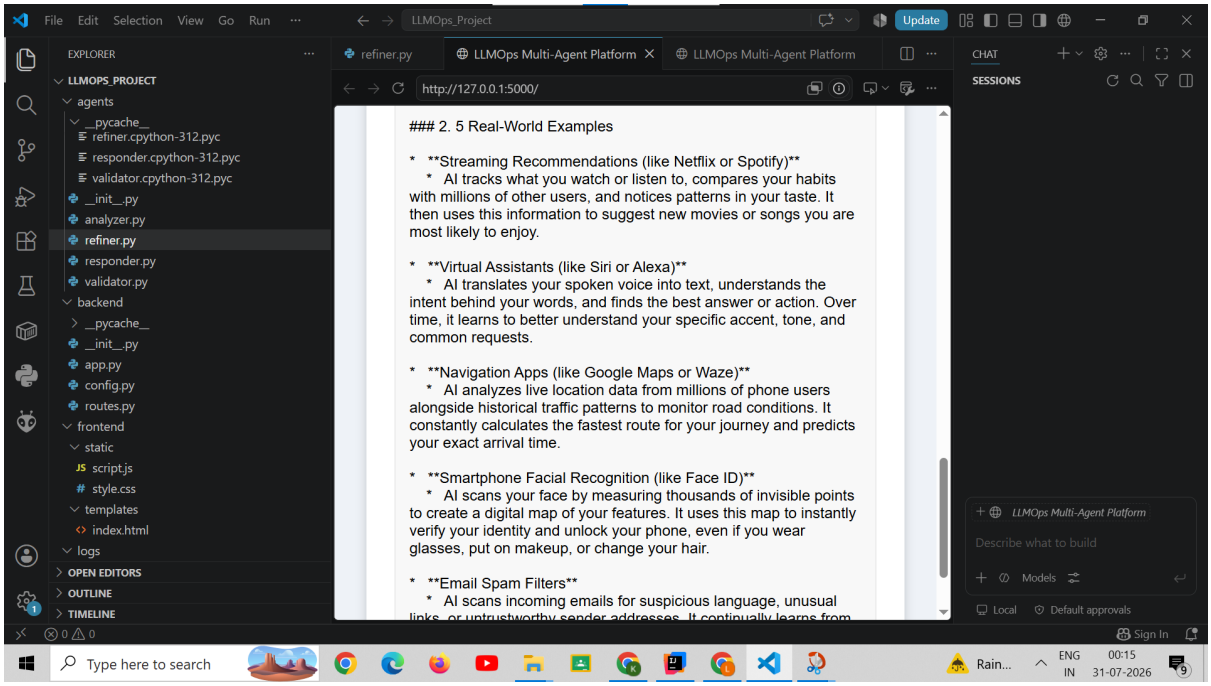


Figure 6.5: Generated AI Response

## 6.2 Discussion

The screenshots confirm that the developed system successfully validates user prompts, analyzes their intent, refines the prompts using a multi-agent architecture, and generates accurate responses using the Google Gemini model. The modular architecture improves prompt quality and enhances the overall effectiveness of AI-generated responses.

# 7. Conclusion

The LLMOps Multi-Agent Platform for Prompt Refinement using Google Gemini successfully demonstrates how multiple intelligent agents can collaborate to improve user prompts before they are processed by a Large Language Model.

The Validation Agent, Analysis Agent, and Refinement Agent work together to produce clear, grammatically correct, and context-aware prompts. The refined prompts enable the Gemini model to generate more relevant and meaningful responses.

The modular architecture improves maintainability, scalability, and future extensibility while reducing the effort required from users to write effective prompts.

## 7.1 Future Scope

The project can be extended in several ways:

- Support multiple Large Language Models.
- Introduce multilingual prompt refinement.
- Add user authentication and profile management.
- Implement cloud deployment.
- Integrate analytics dashboards for prompt performance.
- Support document-based question answering using vector databases.

# 8. References


1. Google Gemini API Documentation
2. Flask Documentation
3. Python Documentation
4. HTML5 Documentation
5. CSS Documentation
6. JavaScript Documentation
7. Prompt Engineering Guide
8. LangChain Documentation
9. CrewAI Documentation
10. GitHub Documentation

# A. Source Code

This appendix contains the major source files used in the implementation of the project.

## A.1 Backend

```
from flask import Flask, render_template, request, jsonify
from dotenv import load_dotenv
from google import genai
import os
import time

# Import AI Agents
from agents.validator import validate_prompt
from agents.analyzer import analyze_prompt
from agents.refiner import refine_prompt
from agents.responder import generate_response

# Import Logger
from logs.logger import save_log

# Load Environment Variables
load_dotenv()

# Create Flask App
app = Flask(
    __name__,
    template_folder="../frontend/templates",
    static_folder="../frontend/static"
)

# Create Gemini Client
client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)


@app.route("/")
```

```
33 def home():
34     return render_template("index.html")
35
36
37 @app.route("/generate", methods=["POST"])
38 def generate():
39
40     try:
41
42         start_time = time.time()
43
44         # Get user prompt
45         data = request.get_json()
46         prompt = data.get("prompt", "")
47
48         # ------------------------------
49         # Agent 1 : Validator
50         # ------------------------------
51         is_valid, validation_message = validate_prompt(prompt)
52
53         if not is_valid:
54             return jsonify({
55                 "validation": validation_message,
56                 "response": validation_message
57             })
58
59         # ------------------------------
60         # Agent 2 : Analyzer
61         # ------------------------------
62         analysis = analyze_prompt(prompt)
63
64         # ------------------------------
65         # Agent 3 : Refiner
66         # ------------------------------
67         refined_prompt = refine_prompt(prompt)
68
69         # ------------------------------
70         # Agent 4 : Responder
71         # ------------------------------
72         answer = generate_response(client, refined_prompt)
73
74         # Calculate execution time
75         execution_time = round(time.time() - start_time, 2)
76
77         # Save Log
78         save_log(
79             prompt=prompt,
```

```
            analysis=analysis,
            refined_prompt=refined_prompt,
            response=answer,
            execution_time=execution_time
        )

        return jsonify({
            "validation": validation_message,
            "analysis": analysis,
            "refined_prompt": refined_prompt,
            "response": answer,
            "execution_time": f"{execution_time} seconds"
        })

    except Exception as e:

       print("Error:", e)

    return jsonify({
        "validation": "Failed",
        "analysis": "",
        "refined_prompt": "",
        "execution_time": "",
        "response": "Unable to connect to Gemini API. Please try again
           later."
    }), 500

if __name__ == "__main__":
    app.run(debug=True)
```

```
import os
from dotenv import load_dotenv

load_dotenv()

GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

## A.2 Agents

```
def analyze_prompt(prompt):
    """
    Analyze the user's prompt and identify its category.
    """

```

```
    prompt_lower = prompt.lower()

    if any(word in prompt_lower for word in ["python", "java", "c++", "code", "program"]):
        return "Programming"

    elif any(word in prompt_lower for word in ["summarize", "summary"]):
        return "Summarization"

    elif any(word in prompt_lower for word in ["translate", "translation"]):
        return "Translation"

    elif any(word in prompt_lower for word in ["calculate", "solve", "equation", "math"]):
        return "Mathematics"

    elif any(word in prompt_lower for word in ["essay", "story", "poem"]):
        return "Creative Writing"

    else:
        return "General Knowledge"
```

```
from google import genai
from dotenv import load_dotenv
import os

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GEMINI_API_KEY")
)

def refine_prompt(prompt):
    """
    Refine the user's prompt using Gemini.
    """

    response = client.models.generate_content(
        model="gemini-3.6-flash",
        contents=f"""
You are an expert Prompt Engineer.

Your job is to rewrite the user's prompt to make it clearer,
more detailed, and more effective for an AI model.
```

```
23
24 Rules:
25 - Keep the original meaning.
26 - Improve clarity.
27 - Add missing context if appropriate.
28 - Do not answer the prompt.
29 - Only return the improved prompt.
30
31 User Prompt:
32 {prompt}
33 """
34     )
35
36     return response.text
```

```
1 def generate_response(client, prompt):
2
3     response = client.models.generate_content(
4         model="gemini-3.6-flash",
5         contents=prompt
6     )
7
8     return response.text
```

```
1 def validate_prompt(prompt):
2     if prompt is None:
3         return False, "Prompt is empty."
4
5     prompt = prompt.strip()
6
7     if len(prompt) == 0:
8         return False, "Prompt is empty."
9
10    if len(prompt) < 5:
11        return False, "Prompt is too short. Please enter at least 5
              characters."
12
13    return True, "Valid prompt."
```

## A.3 Logging

```
1 from datetime import datetime
2 import os
3
4 LOG_FILE = "logs/log.txt"
```

```
5
6  def save_log(prompt, analysis, refined_prompt, response, execution_time
       ):
7
8      os.makedirs("logs", exist_ok=True)
9
10     with open(LOG_FILE, "a", encoding="utf-8") as file:
11
12         file.write("=" * 80 + "\n")
13         file.write(f"Timestamp      : {datetime.now()}\n")
14         file.write("-" * 80 + "\n")
15
16         file.write(f"Original Prompt:\n{prompt}\n\n")
17
18         file.write(f"Prompt Category:\n{analysis}\n\n")
19
20         file.write(f"Refined Prompt:\n{refined_prompt}\n\n")
21
22         file.write(f"Final Response:\n{response}\n\n")
23
24         file.write(f"Execution Time : {execution_time} seconds\n")
25
26         file.write("=" * 80 + "\n\n")
```

## A.4 Frontend Files

The frontend of the application consists of the following files:

- templates/index.html
- static/style.css
- static/script.js

## A.5 Other Files

```
1  from dotenv import load_dotenv
2  from google import genai
3  import os
4
5  load_dotenv()
6
7  client = genai.Client(
8      api_key=os.getenv("GEMINI_API_KEY")
```

```
)

print("Models supporting generateContent:")
print("----------------------------------")

found = False

for model in client.models.list():
    if "generateContent" in model.supported_actions:
        print(model.name)
        found = True

if not found:
    print("No generateContent model is available.")
```

```
from dotenv import load_dotenv
import os
from google import genai

load_dotenv()

api_key = os.getenv("GEMINI_API_KEY")

print("API key loaded:", bool(api_key))

client = genai.Client(
    api_key=api_key
)

response = client.models.generate_content(
    model="gemini-3.6-flash",
    contents="Explain Artificial Intelligence in 5 simple lines."
)

print(response.text)
```